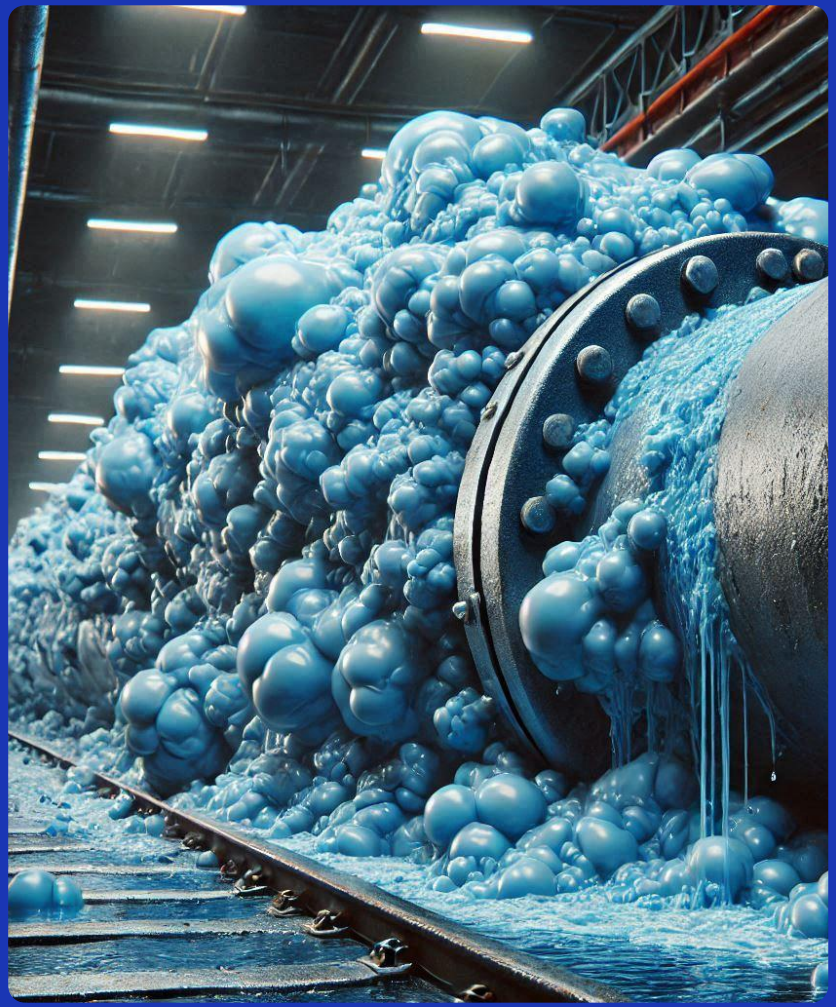
Congestion Pipe Attack

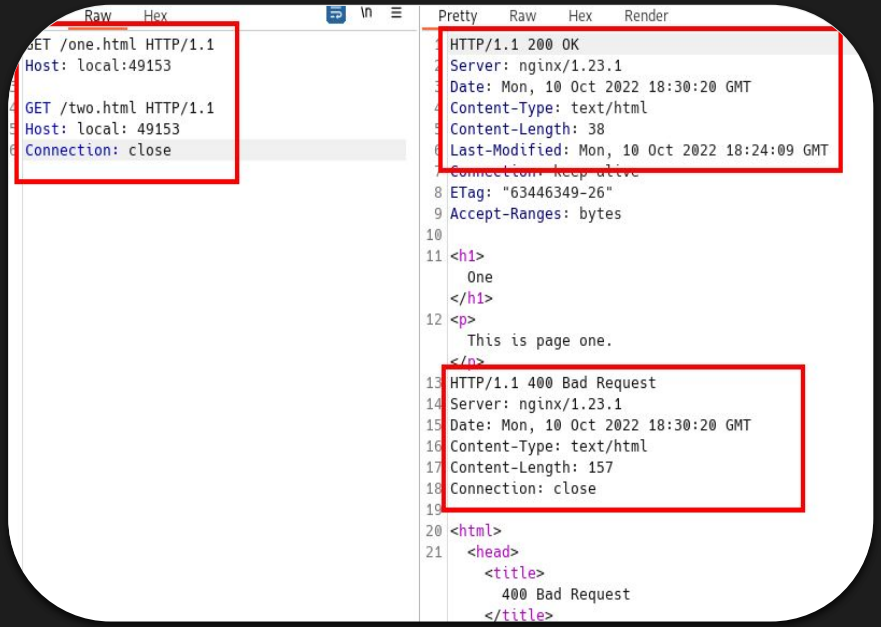
HTTP Pipelining

Alexis Savard



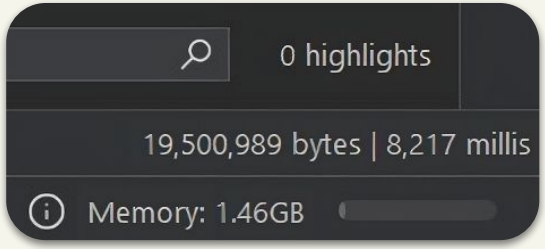
What is HTTP Pipelining

- HTTP pipelining is a feature of HTTP/1.1 that allows a client to send multiple HTTP requests over a single TCP connection without waiting for the corresponding responses.
- Pipelining relies on either the Transfer-Encoding or Content-Length header to be able to tell when one of the pipelined requests stops and the next one begins.
- This technique aims to reduce the latency caused by waiting for each response before sending the next request, which is particularly beneficial in high-latency network conditions.
- Normally used by front-end and back-end communication



How to exploit HTTP Pipelining

When using HTTP pipelining, responses must be returned in the same order as the requests, requiring the server to process them sequentially. If an attacker sends a request requiring lengthy processing, such as a complex database query, all subsequent requests must wait until it's finished, causing delays.



HTTP pipelining can be exploited for Denial of Service (DoS):

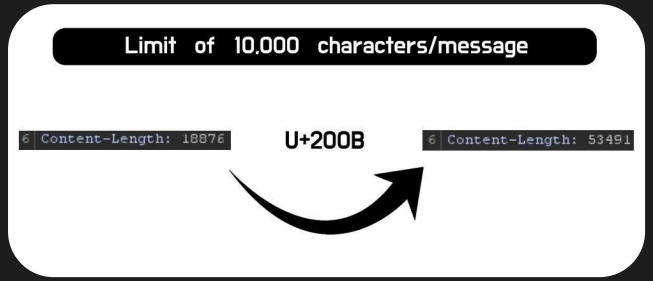
- Attackers can send requests requiring lengthy processing, such as complex database queries or resource-intensive computations.
- Subsequent requests in the pipeline are delayed until the server completes the time-consuming task, leading to degraded performance or service unavailability.

A safe way to test the potential of HTTP pipelining without causing service exhaustion is by gradually adding requests to the main request while monitoring the response times (or alternatively increasing the number of database elements, if applicable). This method helps assess the performance impact of pipelining, showcasing its strain on the server without risking system instability.

Enhancing Request Potential (TIPS)

There are several ways to further exploit this vulnerability. For example, if the system is fetching data from a database, you could explore whether it's possible to include additional elements or manipulate the input to increase the processing load. Depending on the system's constraints, such as character limits or input handling, different techniques could be used to maximize the impact. Here are a few possibilities:

- **Include Additional Data:** If the database query allows, try including more elements to expand the response and processing time.
- **Inflate Input Size:** If there's no character limit, increase the size of each input element to amplify the load during data retrieval.
- **Utilize Unicode Characters:** When a character limit is enforced, use Unicode characters, which can expand significantly after being URL encoded by the server.



- **Request Data Reordering:** Ask the server to reorder the data based on specific criteria, such as sorting by a complex parameter, to add further processing overhead.
- **Explore Other Processing Overheads:** Identify additional ways to increase the server's workload, such as manipulating nested queries or requesting complex operations.

Articles

- https://blog.apnic.net/2022/11/18/service-exhaustion-floods-http-https-flood-http-pipelining-and-ssl-renegotiation-ddos-attack/
  - “Google’s researchers said that the attack on the Google Cloud Armor customer was the equivalent of getting all the daily requests to Wikipedia in just 10 seconds and the attack continued for 69 minutes.”

- https://www.infoworld.com/article/2294811/researcher-devises-hard-to-detect-dos-attack-against-http-servers.html